

UNAM

Universidad Nacional Autónoma de México

Facultad de Ciencias

Modelado y Programación

Práctica 01

Equipo Polimórfo

Juárez Larrondo Omar Alejandro
(322245244)

Soto Mendoza Ismael de Jesús
(322210422)

Vega Navas Saúl (322088267)

Identificación de la Práctica

Este documento corresponde a la **Práctica 01** de la materia *Modelado y Programación* de la Facultad de Ciencias, UNAM. El objetivo principal es implementar adecuadamente los patrones de diseño **Strategy** y **Observer** en la resolución de un problema de simulación de servicios de streaming.

1.1. Descripción del Problema

Se implementó una simulación donde seis usuarios (Alicia, Bob, César, Diego, Erika y Fausto) interactúan con cinco servicios de streaming (Memeflix, Momazon Prime Video, Spootify, Thisney+ y HVO Max) durante un período de un año. La simulación modela diferentes tarifas y planes de suscripción para cada servicio, incorporando períodos gratuitos y cambios de tarifa por tiempo transcurrido. Asimismo, el sistema incluye notificaciones automáticas entre servicios y usuarios, un sistema de pagos centralizado mediante cuentas bancarias, y comportamientos específicos de cada usuario a lo largo de los meses.

Análisis de Implementación de Patrones

2.1. Patrón Observer

¿Por qué se eligió esta implementación?

El patrón Observer fue implementado para gestionar la comunicación entre los servicios de streaming (Subject) y los usuarios (Observer) de manera desacoplada. Esta elección se fundamenta en el desacoplamiento que permite a los servicios notificar a múltiples usuarios sin conocer sus implementaciones específicas, proporcionando flexibilidad para agregar o remover usuarios de los servicios dinámicamente, y garantizando la consistencia necesaria para que todos los usuarios suscritos reciban las notificaciones apropiadas.

En cuanto a la implementación concreta, la interfaz *Observer* es implementada por la clase Usuario, mientras que la interfaz *Subject* es implementada por la clase abstracta Servicio. Las notificaciones incluyen información sobre meses de uso, recomendaciones de contenido, mensajes de bienvenida y despedida, así como los resultados de las operaciones de cobro.

2.2. Patrón Strategy

¿Por qué se eligió esta implementación?

El patrón Strategy fue implementado para encapsular los diferentes algoritmos de cobro de cada servicio. Esta decisión se basa en la variabilidad de tarifas que presenta cada servicio, ya que cada uno tiene lógicas de cobro únicas que incluyen tarifas fijas, variables por tiempo y períodos gratuitos. El patrón también proporciona intercambiabilidad, permitiendo que los usuarios cambien de plan sin afectar la estructura del sistema, y ofrece extensibilidad al facilitar la adición de nuevos planes sin requerir modificaciones al código existente.

Respecto a la implementación concreta, la interfaz *Strategy* está representada por *EstrategiaCobro*, la cual define métodos para calcular costos y obtener descripciones de planes. Las *ConcreteStrategies* corresponden a las implementaciones específicas para cada plan de cada servicio, mientras que el *Context* está encarnado por la clase *HistorialServicio*, que utiliza las estrategias correspondientes para procesar los cobros de manera apropiada.

Estado de la Implementación

3.1. Componentes Implementados

La práctica se encuentra **completamente implementada** con todos los componentes requeridos. El desarrollo incluye la correcta implementación de los patrones Observer y Strategy, la definición de seis usuarios con comportamientos específicos, la configuración de cinco servicios de streaming con sus

tarifas correspondientes, y un sistema de cobros centralizado que opera mediante cuentas bancarias. Adicionalmente, se ha completado una simulación temporal de 12 meses, se genera reportes detallados en archivos de texto, y se proporciona documentación Javadoc en todas las clases públicas del sistema.

3.2. Estructura del Proyecto

El proyecto está organizado arquitectónicamente en varios paquetes especializados. El paquete `patrones` contiene las interfaces fundamentales `Observer`, `Subject` y `EstrategiaCobro`. El paquete `modelo` incluye las clases centrales `Usuario`, `CuentaBanco`, `HistorialServicio` y `ResultadoCobro`. La jerarquía completa de servicios de streaming se encuentra en el paquete `servicios`, mientras que las implementaciones concretas de los algoritmos de cobro residen en `estrategias`. La orquestación temporal del sistema se maneja desde el paquete `simulacion`, y finalmente, la gestión de archivos y la generación de reportes se centraliza en el paquete `utilidades`.

Funcionamiento de la Práctica

La simulación opera mediante un sistema complejo que orquesta la interacción temporal entre usuarios, servicios y sistemas de pago durante un ciclo de doce meses. El núcleo del funcionamiento reside en la clase `Simulacion`, que coordina meticulosamente cada aspecto de la experiencia de streaming de los seis usuarios participantes. Este proceso inicia con la configuración automática de usuarios, donde cada individuo (Alicia, Bob, César, Diego, Erika y Fausto) se inicializa con su presupuesto específico y sus correspondientes cuentas bancarias, mientras que simultáneamente se establecen los cinco servicios de streaming con sus respectivas estrategias de cobro y catálogos de recomendaciones mensuales.

El flujo de ejecución mensual sigue una secuencia estructurada que garantiza la coherencia temporal del sistema. Durante cada mes, el sistema aplica primero los comportamientos específicos predefinidos para cada usuario, ejecutando las decisiones de suscripción, cancelación o cambio de plan que corresponden al período temporal actual. Estas operaciones modifican dinámicamente las listas de observadores de cada servicio, activando o desactivando las relaciones de notificación según las suscripciones vigentes. Posteriormente, se ejecuta la fase de procesamiento de cobros mensuales, donde cada servicio solicita el pago correspondiente a todos sus usuarios activos, utilizando las estrategias de cobro apropiadas según el plan contratado y el número de meses transcurridos desde la suscripción inicial.

La implementación del patrón `Strategy` se manifiesta de manera particularmente sofisticada en el sistema de cobros. Cada servicio utiliza estrategias específicas que encapsulan algoritmos de cálculo únicos, permitiendo el manejo de tarifas variables sin comprometer la flexibilidad del sistema. Las estrategias de `Thisney+` implementan una lógica escalonada que aplica una tarifa promocional de 130 pesos durante los primeros tres meses, incrementándose automáticamente a 160 pesos a partir del cuarto mes. De manera similar, `HVO Max` ejecuta una estrategia que proporciona acceso completamente gratuito durante el período inicial de tres meses, activando posteriormente un cobro de 140 pesos mensuales. Las estrategias de `Memeflix` manejan diferentes niveles de servicio basados en el número de dispositivos simultáneos, mientras que `Momazon` diferencia entre planes normales y premium, y `Spotify` administra tanto versiones gratuitas como de pago.

El patrón `Observer` se despliega a través de un sistema integral de notificaciones que mantiene informados a los usuarios sobre todos los aspectos relevantes de sus suscripciones. Los servicios, actuando como sujetos observables, envían automáticamente notificaciones de bienvenida cuando se establece una nueva suscripción, distinguiendo entre usuarios completamente nuevos y aquellos que regresan después de cancelaciones previas. Mensualmente, cada servicio notifica a sus suscriptores activos el número acumulativo de meses de uso, reforzando la continuidad de la relación comercial. Las recomendaciones de contenido se distribuyen dinámicamente, con cada servicio proporcionando sugerencias específicas que varían mes a mes para mantener el engagement de los usuarios.

La gestión financiera del sistema se centraliza a través de las instancias de `CuentaBanco`, que procesan todas las transacciones de manera unificada y mantienen un historial completo de operaciones. Cuando un servicio solicita un cobro, la cuenta bancaria verifica la disponibilidad de fondos, procesa el pago si es posible, y coordina las notificaciones correspondientes tanto al usuario como al servicio solicitante.

En casos de fondos insuficientes, el sistema ejecuta automáticamente la cancelación de la suscripción, enviando notificaciones de despedida y removiendo al usuario de la lista de observadores del servicio afectado.

Una característica crucial del sistema es la persistencia del estado de suscripciones a través de la clase `HistorialServicio`. Esta implementación mantiene contadores acumulativos de meses contratados que se preservan incluso después de cancelaciones, evitando que los usuarios puedan aprovechar repetidamente los períodos gratuitos o promocionales de servicios como HVO Max y Thisney+. Esta funcionalidad garantiza la integridad económica de la simulación y refleja el comportamiento esperado en sistemas comerciales reales.

Los comportamientos individuales de cada usuario se ejecutan de manera diferenciada según patrones predefinidos que reflejan diferentes estrategias de consumo de servicios de streaming. Alicia representa el perfil de usuario premium que mantiene suscripciones a todos los servicios con los planes más costosos disponibles durante toda la simulación. Bob ilustra un patrón de sobrestimación inicial seguido de cancelaciones progresivas debido a limitaciones presupuestarias. César adopta una estrategia conservadora inicial con expansión gradual, mientras que Diego ejecuta un patrón complejo de suscripciones escalonadas con cambios de plan. Erika demuestra un comportamiento cíclico de suscripciones y cancelaciones masivas, y Fausto presenta un patrón de alternancia constante entre diferentes servicios.

La generación de reportes constituye la fase final del procesamiento mensual, donde el sistema documenta exhaustivamente todas las transacciones, cambios de estado y notificaciones ocurridas. El gestor de archivos produce múltiples tipos de reportes que incluyen resúmenes mensuales detallados, estados financieros finales de todos los usuarios, y un registro completo de transacciones que permite el análisis posterior del comportamiento del sistema. Esta documentación garantiza la trazabilidad completa de todas las operaciones realizadas durante la simulación.

El sistema mantiene coherencia temporal mediante la sincronización cuidadosa de todas sus operaciones, asegurando que las notificaciones, cobros y cambios de estado ocurran en el orden apropiado para cada mes simulado. Esta coordinación permite que los patrones de diseño implementados funcionen de manera armoniosa, proporcionando una experiencia de simulación realista que refleja fidedigamente las complejidades de los ecosistemas de servicios de streaming contemporáneos.

Requisitos del Sistema

5.1. Versión de Java

La implementación de esta práctica requiere específicamente **Java Development Kit (JDK) versión 17** o superior para su correcta compilación y ejecución. Esta elección tecnológica se fundamenta en las características avanzadas que Java 17 proporciona como versión de soporte a largo plazo, garantizando estabilidad y compatibilidad extendida para el desarrollo académico y profesional. Java 17 introduce mejoras significativas en el rendimiento de la máquina virtual, optimizaciones en la gestión de memoria y refinamientos en las estructuras de datos fundamentales que benefician directamente el comportamiento de los patrones de diseño implementados en este proyecto.

La versión seleccionada ofrece características particulares que resultan especialmente ventajosas para la arquitectura del sistema de simulación. Las mejoras en la gestión de colecciones facilitan el manejo eficiente de las listas de observadores en el patrón `Observer`, mientras que las optimizaciones en la creación y destrucción de objetos benefician significativamente el procesamiento de las múltiples instancias de estrategias de cobro del patrón `Strategy`. Adicionalmente, Java 17 proporciona mayor robustez en el manejo de excepciones y una gestión más eficiente de los recursos del sistema, aspectos cruciales para el procesamiento de las transacciones financieras simuladas y la generación de reportes detallados.

Las funcionalidades específicas de Java 17 también permiten un mejor aprovechamiento de las capacidades del sistema operativo subyacente, resultando en una ejecución más fluida de la simulación temporal que abarca los doce meses de operación. Esta versión garantiza compatibilidad con las bibliotecas estándar utilizadas para la manipulación de archivos y el procesamiento de cadenas de texto, elementos

fundamentales para la generación de los reportes de transacciones requeridos por la especificación de la práctica.

Compilación y Ejecución del Proyecto

6.1. Proceso de Compilación

La compilación del proyecto se realiza mediante el compilador estándar de Java `javac`, aprovechando la estructura modular de paquetes que organiza las clases según sus responsabilidades funcionales. El proceso requiere la navegación al directorio raíz del proyecto, específicamente a la carpeta que contiene el directorio `src`, desde donde se ejecutan los comandos de compilación. La arquitectura del sistema demanda una compilación ordenada que respete las dependencias entre las diferentes clases y paquetes, comenzando por las interfaces fundamentales y progresando hacia las implementaciones concretas.

El comando de compilación debe ejecutarse posicionándose en el directorio principal del proyecto, utilizando la instrucción `javac -cp . -d . src/**/*.java` para compilar recursivamente todas las clases Java contenidas en la estructura de paquetes. Este enfoque garantiza que todas las dependencias entre clases se resuelvan correctamente, generando los archivos `.class` correspondientes en la estructura de directorios apropiada que replica la organización de paquetes del código fuente. La compilación procesa sistemáticamente las interfaces del paquete `patrones`, las clases modelo del paquete `modelo`, la jerarquía de servicios del paquete `servicios`, las estrategias concretas del paquete `estrategias`, la lógica de simulación del paquete `simulacion`, y las utilidades del paquete `utilidades`.

Durante el proceso de compilación, el sistema verifica la sintaxis de todas las clases, valida las relaciones de herencia e implementación de interfaces, y confirma que todas las referencias entre clases sean correctas y accesibles. Este proceso genera una estructura de directorios de clases compiladas que mantiene la misma organización jerárquica que el código fuente, permitiendo que la máquina virtual de Java localice y cargue correctamente todas las clases durante la ejecución. La compilación exitosa produce archivos `bytecode` optimizados que aprovechan las características específicas de Java 17 para proporcionar un rendimiento óptimo durante la simulación.

6.2. Ejecución del Sistema

La ejecución del sistema se inicia mediante el comando estándar de Java `java`, especificando la clase principal `Main` que actúa como punto de entrada de la simulación completa. Desde el directorio raíz del proyecto, donde se encuentran los archivos `.class` compilados, se ejecuta la instrucción `java Main` para iniciar el procesamiento de la simulación temporal. Este comando activa la máquina virtual de Java, carga todas las clases necesarias según las dependencias del sistema, e inicia la ejecución secuencial de los doce meses de simulación con todos los comportamientos específicos de los usuarios y servicios.

Durante la ejecución, el sistema presenta información de progreso en la consola, mostrando las actividades principales de cada mes simulado, incluyendo las suscripciones, cancelaciones, cambios de plan, y procesamiento de cobros. La salida en tiempo real permite supervisar el funcionamiento correcto de los patrones `Observer` y `Strategy`, evidenciando las notificaciones automáticas entre servicios y usuarios, así como la aplicación dinámica de las diferentes estrategias de cobro según el contexto específico de cada suscripción. El programa ejecuta completamente la simulación de los doce meses, procesando las transacciones correspondientes a cada usuario según sus patrones de comportamiento predefinidos.

Al finalizar la simulación, el sistema genera automáticamente archivos de texto que contienen reportes detallados de todas las transacciones realizadas durante el período completo. Estos archivos se crean en el directorio de ejecución y contienen información estructurada sobre cada operación financiera, cambio de suscripción, y notificación enviada, proporcionando un registro completo y auditable de toda la actividad simulada. La información se presenta en formato legible que facilita el análisis posterior del comportamiento del sistema y la verificación de la correcta implementación de los requisitos especificados en la práctica.